

Introduction to Computer Simulation

Home Work 5

Guilherme Schewtschik (3748052)

November 26, 2025

Question 1

The following code was used for the plots:

```
def get_Xi(i, X0 = 1, a = 5, c = 0, m = 2048):
    X = np.zeros(i+1)
    X[0] = X0
    for j in range(i):
        X[j+1] = (a*X[j]+c)%m

    X_max = np.max(X)

    return X/X_max

def get_sigma(X,N):
    X_sm = np.sum(X**2)/(N)
    X_ms = (np.sum(X)/(N))**2

    return X_sm - X_ms

def get_running_average(X,N_max):
    X_r_avg = np.zeros(N_max)

    for i in range(1,N_max):
        X_r_avg[i] = np.sum(X[:i])/i

    return X_r_avg, X_r_avg[-1]

X = get_Xi(10**5, a = 273673163155, c=11, m=2**48)

N1 = 10**3
N2 = 10**4
N3 = 10**5

sigma1 = get_sigma(X[:N1],N1)
sigma2 = get_sigma(X[:N2],N2)
sigma3 = get_sigma(X[:N3],N3)

Xavg1 = get_running_average(X,N1)
Xavg2 = get_running_average(X,N2)
Xavg3 = get_running_average(X,N3)

sigma = [sigma1, sigma2, sigma3]
Xavg = [Xavg1, Xavg2, Xavg3]

t1 = np.arange(N1)
t2 = np.arange(N2)
t3 = np.arange(N3)
```

Which gives us the graphs:

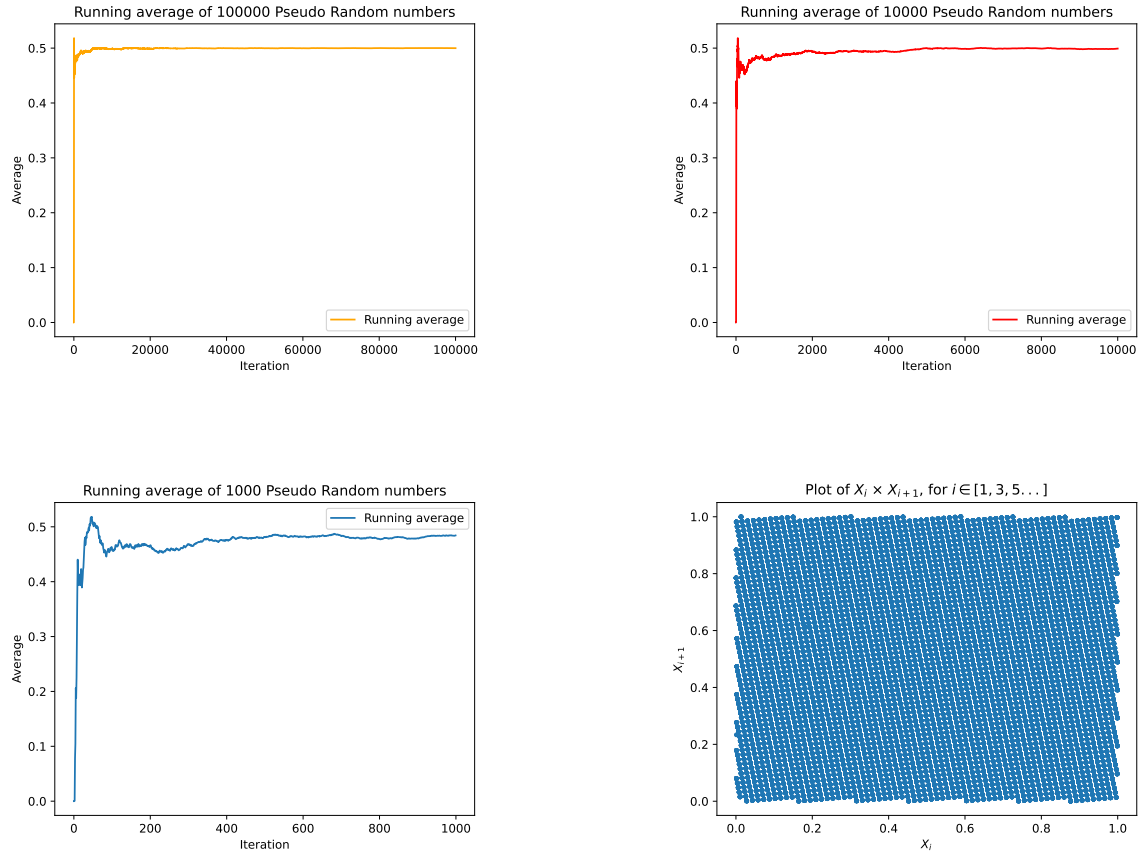


Figure 1: Overall caption for all 4 figures

Question 2

Editing the function from the previous homeworks to include periodic boundary conditions and the Glauber Algorithm we get:

```
def get_Energy_state_periodic(state,J, h=0):

    Energy_interaction = np.zeros(len(state))
    Magnetization = np.sum(h*state)

    for i in range(0,len(state)):
        Energy_interaction[i] = state[i-1]*state[i]

    Energy_total = -J*Energy_interaction.sum() - Magnetization

    return Energy_total, -J*Energy_interaction.sum() , Magnetization

def time_Evolution_Glauber(state,J,h,beta,iterations, get_Energy_state):
    state_mu = state.copy()
    state_evolution = np.zeros((iterations, len(state)))
    net_Energy = np.zeros(iterations)
    net_Energy[0] = get_Energy_state(state,J,h)[0]

    for i in range(iterations):
        x = np.random.randint(0,len(state_mu))
        S = state_mu[(x-1)%len(state_mu)] + state_mu[(x+1)%len(state_mu)]
```

```

deltaE = 2*state_mu[x]*S
p = 1/(1+np.exp(deltaE*beta))

if (np.random.random()<=p):
    state_mu[x] = -state_mu[x]

state_evolution[i] = state_mu
net_Energy[i] = get_Energy_state(state_mu,J,h)[0]

return (state_evolution, net_Energy, get_Energy_state,
        get_Energy_state(state_evolution[-1],J,h)[0])

```

The observables can be calculated through:

```

C = beta**2 * (E2_mean - E_mean**2) / N
chi = beta * N * (M2_mean - M_mean**2) / N**2

```

Due to the long run time only the graph for $N = 1000$ could be calculated. Which gives us the following graph:

